Self-Supervised Learning on Cholec80: Phase Recognition and Tool Presence Detection

Ferial Delavari

September 4, 2026

1 Introduction

This project addresses Topic 2 of the assessment: train a self-supervised (SSL) backbone on Cholec80, then evaluate it on **phase recognition** and **tool presence detection**. Labels may only be used in two ways: (1) to train a classifier on top of the frozen backbone, and (2) to fine-tune the backbone with LoRA.

2 Dataset

Cholec80 [1] contains 80 laparoscopic cholecystectomy videos (13 surgeons, 25 fps), annotated with 7 phase labels (per frame) and 7 tool-presence labels (per second): Preparation, Calot Triangle Dissection, Clipping and Cutting, Gallbladder Dissection, Gallbladder Packaging, Cleaning and Coagulation, Gallbladder Retraction; Grasper, Bipolar, Hook, Scissors, Clipper, Irrigator, Specimen Bag.

The full dataset is ∼76 GB, too large for free Colab storage. I used `remotezip` to stream only the files I needed, and selected **16 videos** (every 5th video, `video01, video06, ..., video76`) for surgeon diversity within Colab's storage/time budget.

3 Data Preparation

Phase labels exist at 25 fps, tool labels at 1 fps; I aligned both at **1 fps** (the limiting rate, matching most published baselines). For each video I matched tool-labeled frames to their phase label, extracted the frame at 224×224, and logged everything to one CSV. This gave **40,357 labeled frames** with zero extraction failures.

I split by **whole video**, not frame (∼70/15/15 train/val/test), since frames from the same video are highly correlated; a frame-level split risks near-duplicates leaking across train and test.

4 Exploratory Data Analysis

Phase imbalance is ∼16:1 (longer surgical steps naturally produce more frames). Tool imbalance is far worse, ∼30–60:1: Grasper/Hook dominate, while Scissors/Bipolar/Clipper/Irrigator each appear in only 2–6% of frames. Because of this I use class-weighted/focal loss and report macro-F1 rather than accuracy for tools, since accuracy alone would look good even if the model always predicted "absent." These proportions are estimates from my 16-video sample, not the full 80-video population.

Phase	Frame Count
Calot Triangle Dissection	17,417
Gallbladder Dissection	12,518
Clipping and Cutting	3,836
Preparation	2,180
Cleaning and Coagulation	1,695
Gallbladder Packaging	1,651
Gallbladder Retraction	1,060

Table 1: Frame count per phase.

Tool	Train (%)	Val (%)	Test (%)
Grasper	60.54	37.36	57.44
Hook	56.90	60.99	59.90
Specimen Bag	4.29	8.22	5.22
Clipper	3.83	2.90	3.90
Irrigator	4.55	2.80	4.49
Bipolar	2.39	2.80	5.81
Scissors	1.79	0.89	1.93

Table 2: Percentage of frames containing each tool, by split.

5 Self-Supervised Pretraining (DINO)

5.1 Method

DINO [2] trains a **student** and **teacher** network of identical architecture with no labels: an image is turned into 2 *global* crops (fed only to the teacher) and 6 *local* crops (fed only to the student), and the student is trained to match the teacher's output distribution over K=2048 pseudo-categories. Both networks' raw outputs are converted to probabilities via a temperature-scaled softmax, $P(x)^{(i)} = \exp(g_\theta(x)^{(i)}/\tau)/\sum_k \exp(g_\theta(x)^{(k)}/\tau)$, with a sharper teacher temperature so the student chases a confident target. The student minimizes cross-entropy $\mathcal{L} = -\sum_i P_t(x_t)^{(i)} \log P_s(x_s)^{(i)}$ via backprop; the teacher is never updated by gradient descent, only as an EMA of the student, $\theta_t \leftarrow m\theta_t + (1 - m)\theta_s$, which (together with centering the teacher's outputs) prevents the trivial collapse a naively symmetric setup would be prone to.

5.2 Setup

I used `lightly` [3] for the loss, EMA update, and multi-crop pipeline rather than re-implementing this math by hand, since a small bug in any of these steps (e.g. an undetached teacher gradient) tends to silently collapse training rather than raise an error. Training used PyTorch Lightning, based on `lightly`'s reference DINO example.

No hyperparameter tuning was done at this stage, matching common practice in larger SSL work [4, 5]: pretraining is too expensive to re-run repeatedly for tuning, and pretext loss does not reliably predict downstream quality [6]. Tuning effort was instead spent on the cheap downstream stages. Only `train`-split frames were used for pretraining; val/test videos were never seen, even unlabeled, to prevent the backbone from learning video-specific quirks that could inflate later

Setting	Value
Backbone	ResNet-18 / ViT-S/16 (random init, no ImageNet)
Projection head	$512/384 \to 512 \to 64 \to 2048$
Global / local crops	2 (teacher) / 6 (student)
Teacher temperature	$0.04 \to 0.07$ over 5 epochs
EMA momentum	$0.996 \to 1.0$ (cosine)
Optimizer	Adam, lr = 0.001, batch size 16
Epochs	5–10
Data	unlabeled frames, `train` split only

Table 3: DINO pretraining settings (shared by both backbones tested).

results, consistent with prior surgical SSL work [7].

Figure 1: DINO's crops on one Cholec80 frame (Calot Triangle Dissection). Left to right: original, 2 global crops (teacher), 6 local crops (student). Local crops can contain very little context on their own, which is exactly the pressure that forces the student to learn generalizable features.

6 Downstream Evaluation

6.1 Classifier Probing

The backbone is fully frozen; a single linear layer per task (phase: 7-way softmax; tools: 7 independent sigmoids) is trained on top of the raw 512/384-dim embedding. This is logistic regression, not an MLP, on purpose: it isolates how good the backbone's own features are, since adding classifier capacity would muddy that question. Imbalance is addressed three ways: (1) a weighted random sampler oversamples rare phases, (2) class-weighted cross-entropy for phase and focal loss for tools, (3) per-tool decision thresholds tuned on validation (rather than a flat 0.5, which is usually wrong for a class present in $<2\%$ of frames) and then applied unchanged to test.

Metric (ResNet-18, test)	Value
Phase Accuracy	0.0753
Phase Macro F1	0.0938
Tool Macro F1	0.2927

Table 4: Frozen-probe headline results (per-tool breakdown in Table 5).

Grasper/Hook reach F1 $\sim$0.73–0.75; the rare tools remain weak (Scissors F1=0), matching the documented imbalance. Phase accuracy (7.5%) is *below* the 14.3% random-guessing baseline, likely because the phase sampler, tuned inside the same loop as the tool sampler, oversampled rare phases aggressively enough to skew the phase head's training distribution away from the real, imbalanced evaluation distribution. Using separate samplers per task is a natural fix for future iterations.

6.2 LoRA Fine-Tuning

LoRA adds a small trainable correction $W' = W + BA$ next to each targeted weight matrix W, which stays frozen; only the small A, B add ∼1–3% extra parameters. B starts at zero, so LoRA training begins identical to the frozen probe and only drifts as A, B learn a task-specific adjustment. This avoids the overfitting risk of full fine-tuning on only 16 labeled videos. Adapters (rank 8, α=16) were placed on ResNet-18's deepest conv layers (`layer3/layer4`) or, for ViT, on the attention `qkv/proj` linear layers, the standard place to adapt higher-level, task-relevant features. Fresh classifier heads were trained on top, with the same imbalance handling as the frozen probe, so any difference in results comes from the adapters alone.

7 Backbone Comparison: ResNet-18 vs. ViT-S/16

The full pipeline (SSL, probing, LoRA) was repeated with a ViT-S/16 backbone (loaded from the original DINO authors' implementation, also random-init), keeping the data, split, epochs, and evaluation protocol identical.

	ResNet-18		**ViT-S/16**	
Metric	**Frozen**	**LoRA**	**Frozen**	**LoRA**
Phase Accuracy	0.0753	0.1061	0.0579	0.1113
Phase Macro F1	0.0938	0.1399	0.0590	0.1258
Tool Macro F1	0.2927	0.3485	0.2561	0.2963
Grasper F1	0.7279	0.7408	0.7214	0.7281
Bipolar F1	0.0893	0.0927	0.0979	0.0875
Hook F1	0.7450	0.7371	0.6863	0.7091
Scissors F1	0.0000	0.2160	0.0261	0.0305
Clipper F1	0.0443	0.0845	0.0334	0.0732
Irrigator F1	0.0738	0.1711	0.1105	0.1839
Specimen Bag F1	0.3686	0.3976	0.1169	0.2616

Table 5: ResNet-18 vs. ViT-S/16, frozen probe and LoRA, held-out test split.

LoRA improved every metric over its frozen baseline, for both backbones (largest jump: Scissors F1 0→0.216 with ResNet), the main finding of this project: a small, cheap, label-driven adjustment measurably improves downstream performance even for a lightly-pretrained backbone.

ResNet-18 beat ViT-S/16 on every aggregate metric except LoRA phase accuracy, where ViT edges ahead (0.111 vs. 0.106). This is consistent with CNNs' built-in spatial inductive bias (locality, weight sharing), which a Transformer must instead learn purely from attention over data; under this project's constraints (a small data subset, 5–10 epochs, one T4 GPU, from scratch), that is close to a worst case for a Transformer, so ResNet's advantage here reads as a training-budget effect rather than a general weakness of ViTs, which are known to overtake CNNs given enough data/compute.

8 Interpretability

8.1 ResNet-18: Grad-CAM

Grad-CAM traces backward from a predicted class score to the last convolutional feature map, weighting each spatial location by how much it influenced that prediction, producing a heatmap overlay. This requires one forward and one backward pass only (no optimizer step, so no weights change); the backbone is briefly un-frozen only to allow that one gradient computation, then re-frozen.

All three sampled frames are true Calot Triangle Dissection; the model predicted a different, incorrect phase for all three, consistent with the below-chance accuracy in Section 6.1. Two heatmaps concentrate on the dark vignette border of the endoscopic view rather than tissue/instruments, suggesting the model partly keys off the camera view's shape; the third lands plausibly on the grasper tip and tissue, though the prediction was still wrong.

8.2 ViT-S/16: Attention-Based Visualization

A ViT has no final conv feature map, so the same gradient-weighting idea is applied instead to the 14×14 grid of patch tokens from the last transformer block, reshaped back to spatial layout.

Unlike ResNet's localized hot spots, all three ViT heatmaps are almost perfectly flat and uniform. This ties directly to Section 7: a CNN's convolutions have spatial selectivity built in even when lightly trained, while a Transformer's attention starts (and largely stays, under this little training) close to uniform across patches, so the resulting importance map is close to flat too. The uninformative heatmap and the weak quantitative score are two symptoms of the same root cause, an undertrained Transformer, not two separate problems.

9 Related Work

This project follows CAMMA's SelfSupSurg study [7], which benchmarks DINO (and others) on the full 80-video Cholec80 for the same two tasks using VISSL across multiple GPUs. This is a scaled-down version of that idea: 16 videos, ResNet-18/ViT-S/16, `lightly`, and a single Colab T4.

References

[1] A.P. Twinanda, S. Shehata, D. Mutter, J. Marescaux, M. de Mathelin, N. Padoy. *EndoNet: A Deep Architecture for Recognition Tasks on Laparoscopic Videos.* IEEE Transactions on Medical Imaging (TMI), 2017.

[2] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, A. Joulin. *Emerging Properties in Self-Supervised Vision Transformers.* ICCV, 2021.

[3] Lightly AI. *lightly: A Python library for self-supervised learning.* `https://github.com/lightly-ai/lightly`.

[4] M. Oquab, T. Darcet, T. Moutakanni, et al. *DINOv2: Learning Robust Visual Features without Supervision.* arXiv:2304.07193, 2023.

[5] Z. Wang, C. Liu, S. Zhang, Q. Dou. *Foundation Model for Endoscopy Video Analysis via Large-scale Self-supervised Pre-train (Endo-FM).* MICCAI, 2023.

[6] Anonymous. *DIET-CP: Lightweight and Data Efficient Self Supervised Continued Pretraining.* arXiv, 2025.

[7] CAMMA. *Dissecting Self-Supervised Learning Methods for Surgical Computer Vision (SelfSupSurg).* `https://github.com/CAMMA-public/SelfSupSurg`.

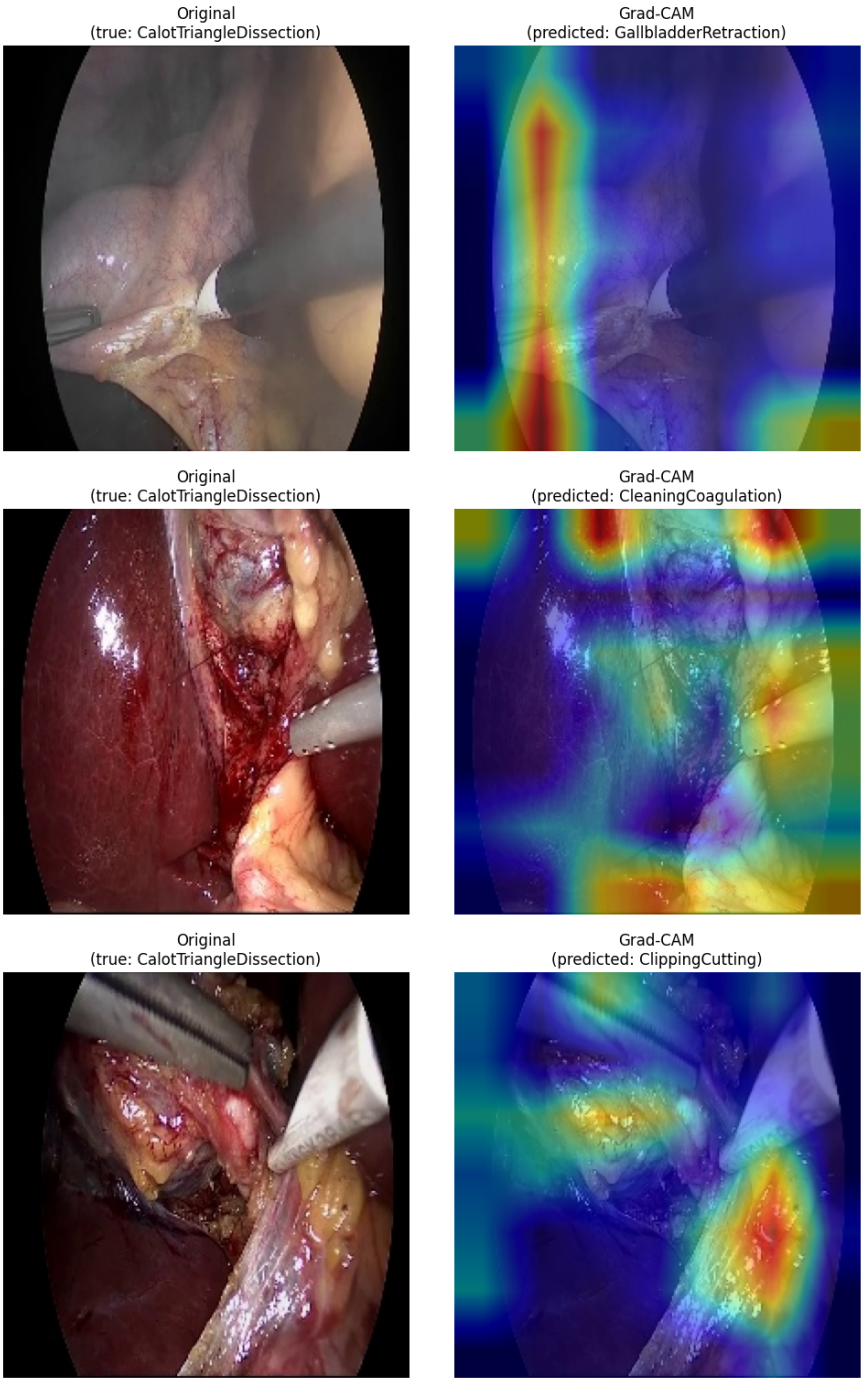

Figure 2: Grad-CAM on 3 test-split frames (original left, heatmap + predicted phase right).

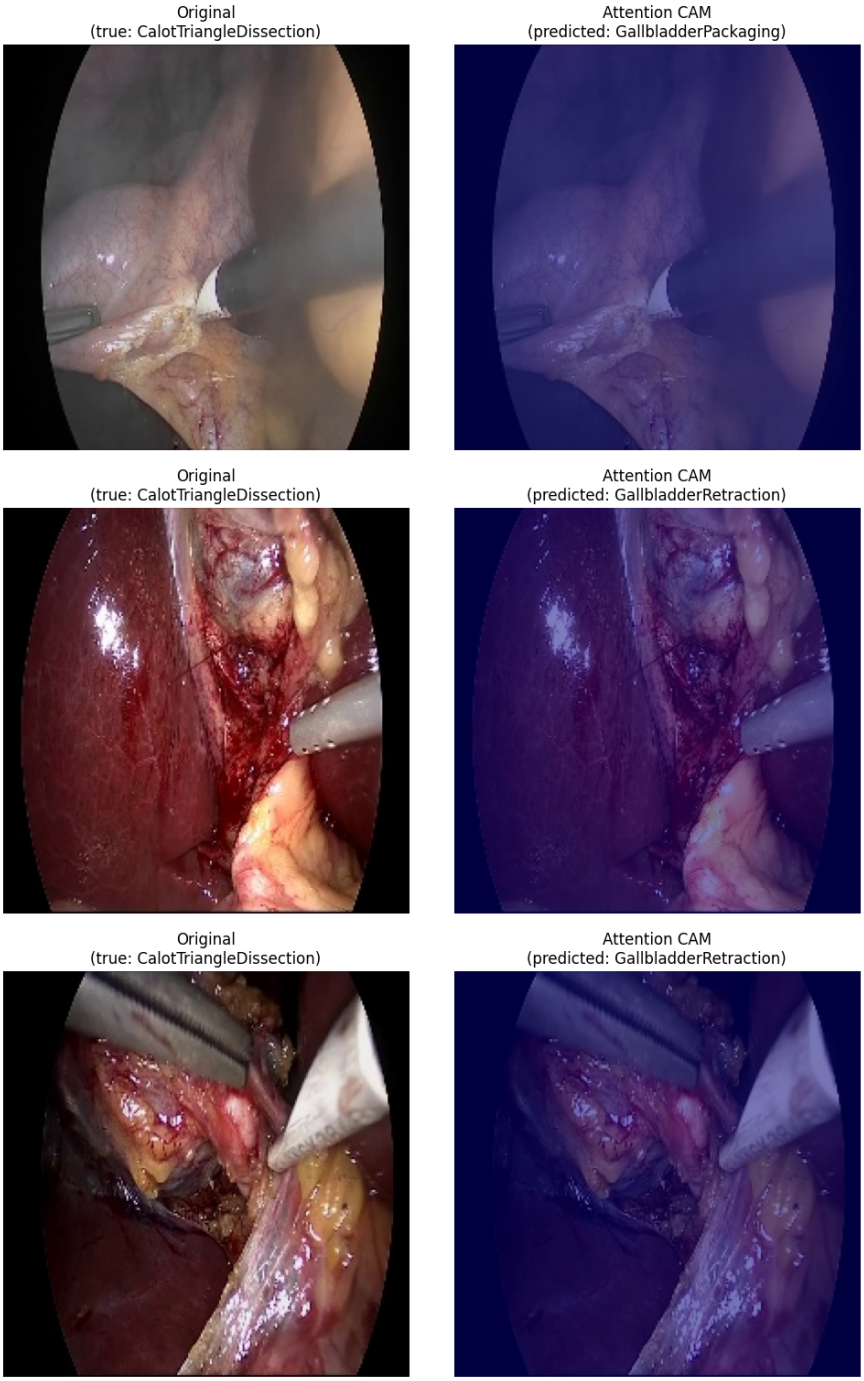

Figure 3: Attention-based heatmaps, same 3 frames as Figure 2.